AGENTIC AI — SEMESTER PROJECT

AI-Powered Animated Video Generation System

From Prompt to Polished Short Film — End-to-End with LLM Agents

Group Size: 4 Members | Course: Agentic AI | Project Type: Full-Stack AI System

Submission Guidelines

- Submission deadline is **5th May (11:59 PM)**. Late submissions will not be accepted without prior approval.
- Each group must consist of **4 members**. Cross-section groups are allowed.
- The project must be submitted by **only one representative from each group** on both GitHub and Google Classroom.
- Each group must create a **GitHub repository** containing the complete project. The repository should include:
  - A well-organized codebase structured according to all project phases.
  - A root README.md file describing the project overview, setup instructions, technology stack, and execution steps.
  - Implementation of all phases:
    - Story and Script Generation (Phase 1)
    - Audio Generation (Phase 2)
    - Video Composition (Phase 3)
    - Web Interface (Phase 4)
    - Edit Agent with Undo Functionality (Phase 5)
  - A requirements.txt (or equivalent dependency file).
  - Sample outputs (audio, images, or video) where applicable.
  - Unit tests for individual phases.
  - Proper commit history demonstrating incremental development.
- The repository must be **public or shared with instructor access**. The naming convention should follow:
  AgenticAI_Project_<GroupName>
- On **Google Classroom**, the submitting member must upload:
  - GitHub repository link
  - Project report (PDF, 8–12 pages)
  - Demo video or screen recording
  - Presentation slides (if prepared)
- The **project report** must include system architecture, phase-wise implementation details, tools and APIs used, JSON schema design, challenges faced, results, and individual contributions.
- The **demo video** (3–7 minutes) must clearly demonstrate the full pipeline from prompt to final video, along with at least three edits and two undo/revert operations through the editing agent.
- All submissions must be original. Any form of plagiarism will result in zero marks.
- Incomplete repositories, missing components, or non-functional pipelines will be penalized.
- Each phase should be modular, properly integrated, and independently testable.
- Students are strongly advised to finalize the shared JSON schema at the beginning, as it is critical for integration across all phases.

1. Project Overview

This project challenges your group to build an end-to-end, agent-orchestrated system that accepts a single natural-language prompt from a user and autonomously produces a complete short animated video — including story, dialogue, character voices, visual scenes, and final composited output — with zero manual creative intervention.

The system is not a simple API wrapper. It is a multi-phase agentic pipeline where each phase is a distinct AI-powered module with defined inputs, outputs, and responsibilities. The four phases map directly to the four group members, and the entire pipeline is unified by a web interface that orchestrates, monitors, and allows re-running of individual phases.

2. Problem Statement

Creating even a short, animated video today requires a team of writers, voice artists, illustrators, animators, and audio engineers — a process that takes days or weeks and costs thousands of dollars. The rise of generative AI models across language, audio, and vision domains makes it theoretically possible to automate this entire pipeline, yet no student-accessible, open-source, end-to-end solution exists that ties all of these components together with an intelligent orchestration layer.

This project solves exactly that problem. Your group will design and implement:

- An LLM-based creative agent that turns a raw user prompt into a structured, scene-by-scene story with full character definitions.
- A voice synthesis module that assigns consistent, character-specific voices and produces a synchronized audio track with background music.
- A visual generation and composition module that produces per-scene imagery, animates it, syncs it with audio, and exports a final MP4.
- A full-stack web application that serves as the user-facing interface and the agent orchestration layer for all pipeline phases.
- An intelligent editing agent that interprets free-text edit commands, detects the target component (audio, video frame, or full video), executes the change, and maintains a versioned state history with full undo support.

3. Learning Objectives

Upon completing this project, each group member will have demonstrated the ability to:

1. Design and implement a multi-agent workflow using LangChain and/or LangGraph.
2. Integrate heterogeneous AI APIs and local models within a single production pipeline.
3. Apply structured output techniques (JSON schema enforcement, prompt chaining) with LLMs.
4. Build a real-time, phase-aware full-stack web application.
5. Handle stateful, versioned data across an agentic pipeline (including undo/revert logic).
6. Collaborate on a modular codebase with clearly defined inter-phase contracts (data schemas).

4. System Architecture

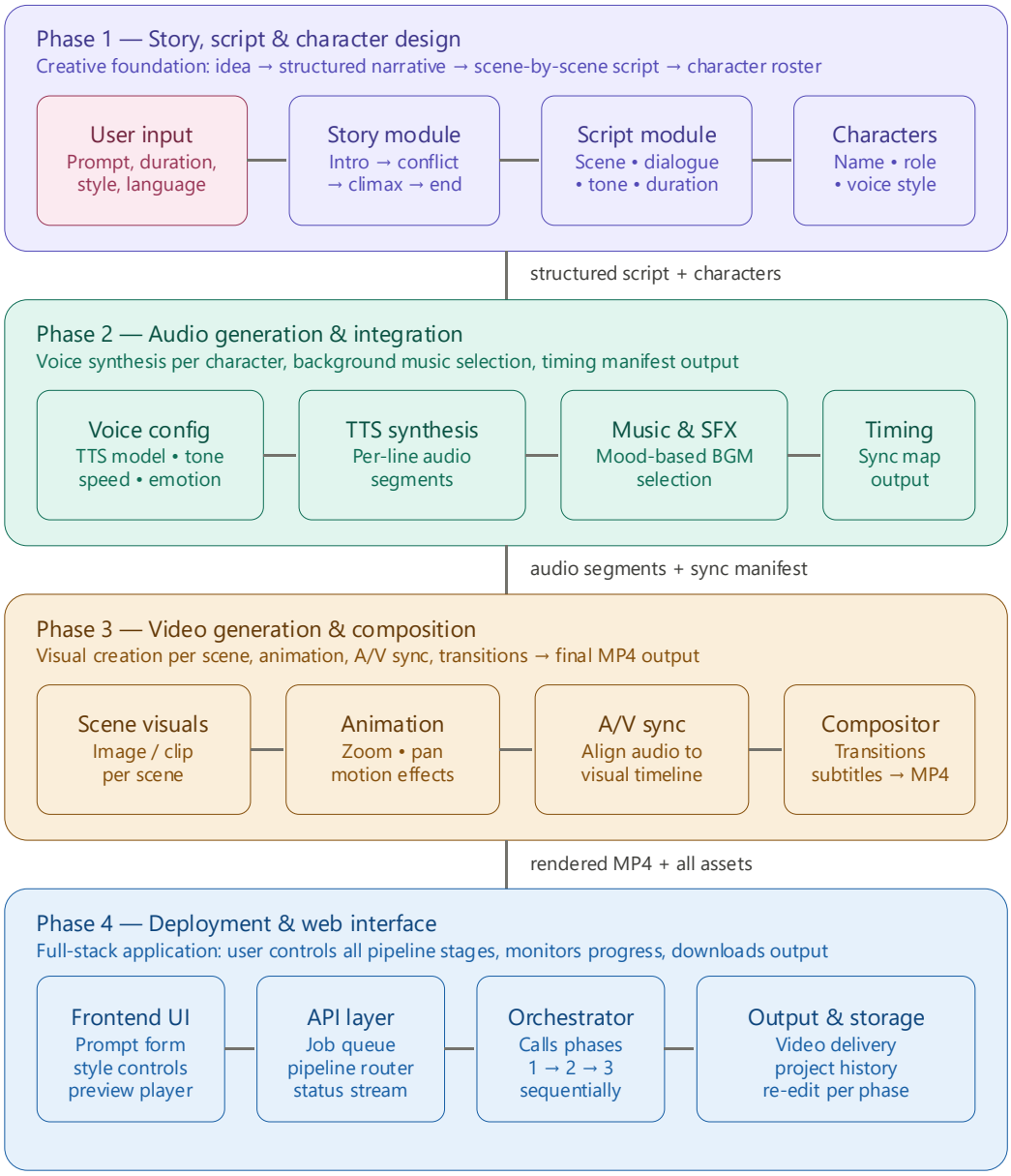
The system consists of five modules. The first four are core pipeline phases; the fifth is the editing agent. All modules communicate through a shared data schema — a central JSON state object that is passed forward, updated, and versioned by each phase.

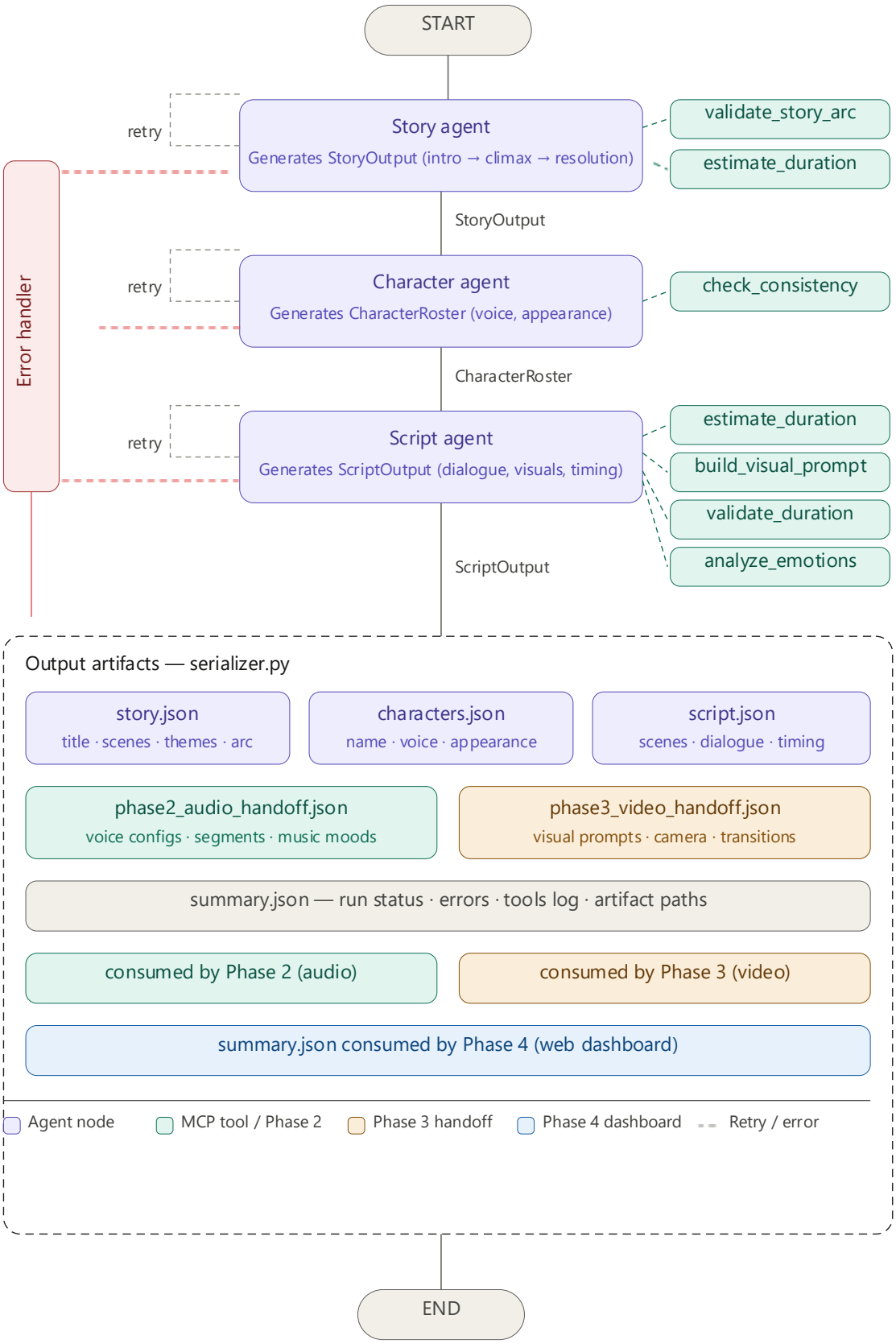
Phase 1: Story, Script & Character Design	
Input	A free-form natural language prompt from the user (e.g., "A young astronaut discovers a hidden ocean on Mars")
LLM Role	Expand prompt → full narrative → scene-by-scene script with dialogue, setting, tone, and duration → character roster with names, roles, voice personalities, and visual descriptions
Output	A validated JSON object: { story, scenes[], characters[] } consumed by all downstream phases
Tools / APIs	LangChain / LangGraph agent; local model via Ollama/llama.cpp OR cloud API (Claude / GPT-4); Pydantic schema validation
Responsibility	Member 1

Phase 2: Audio Generation & Integration	
Input	scenes[] and characters[] from Phase 1 JSON
Tasks	TTS synthesis per dialogue line with per-character voice parameters; background music selection or generation per scene mood; assembly of audio segments into a timing manifest
Output	Audio files (.wav/.mp3) per line + a timing_manifest.json ({ scene_id, audio_file, start_ms, end_ms })
Tools / APIs	ElevenLabs API (cloud) or Coqui TTS / Bark (local); optional MusicGen or royalty-free library for BGM
Responsibility	Member 2

Phase 3: Video Generation & Composition	
Input	scenes[] from Phase 1 JSON + timing_manifest.json from Phase 2
Tasks	Per-scene image generation using prompt-engineered visual descriptions; light animation (zoom/pan/Ken Burns) via FFmpeg or MoviePy; A/V sync using timing manifest; subtitle overlay (optional); compositing all scenes with transitions into final MP4
Output	final_output.mp4 — the finished animated short video
Tools / APIs	Stable Diffusion (local via Automatic1111 / ComfyUI) or DALL-E / RunwayML (API); FFmpeg; MoviePy
Responsibility	Member 3

Phase 4: Deployment & Web Interface	
Input	User prompt (initiates pipeline); phase outputs from Phases 1–3
Tasks	Provide a prompt input UI; display real-time per-phase progress; offer phase-level re-run buttons (e.g., regenerate voice only); serve final video preview and download
Output	A deployed full-stack web application
Tools / APIs	FastAPI or Django (backend); React or Next.js (frontend); WebSocket or SSE for live progress; Docker (optional for deployment)
Responsibility	Member 4




START
Story agent
Generates StoryOutput (intro → climax → resolution)
retry
validate_story_arc
estimate_duration
StoryOutput
Character agent
Generates CharacterRoster (voice, appearance)
retry
check_consistency
CharacterRoster
Script agent
Generates ScriptOutput (dialogue, visuals, timing)
retry
estimate_duration
build_visual_prompt
validate_duration
analyze_emotions
ScriptOutput
Error handler
Output artifacts — serializer.py
story.json
title · scenes · themes · arc
characters.json
name · voice · appearance
script.json
scenes · dialogue · timing
phase2_audio_handoff.json
voice configs · segments · music moods
phase3_video_handoff.json
visual prompts · camera · transitions
summary.json — run status · errors · tools log · artifact paths
consumed by Phase 2 (audio)
consumed by Phase 3 (video)
summary.json consumed by Phase 4 (web dashboard)
Agent node
MCP tool / Phase 2
Phase 3 handoff
Phase 4 dashboard
Retry / error
END

5. Last Phase — Intelligent Edit & Undo System

After the pipeline produces its initial output, the user should be able to describe edits in plain natural language — just as they would instruct a human editor. The system must understand what they want to change, identify which component is the target (audio, video frame, or full video), execute the change, and allow the user to revert to any previous state. For an example you can store collection of filter for images and upon query of use apply that filter or set of filters.

5.1 What the User Can Say

The editing interface is a free-text input field. Examples of valid user queries:

Example User Query	Detected Target	Action Taken
"Change voice tone"	**Audio**	Re-run TTS for the specified character/scene with new tone parameter
"Make the scene darker"	**Video Frame**	Re-generate that scene's visual prompt with updated aesthetic descriptor
"Add background music"	**Audio**	Fetch/generate and overlay new background track for target scene
"Remove the subtitle"	**Video**	Recompose video output without subtitle burn-in
"Change character design"	**Video Frame**	Re-run image generation for all frames featuring that character
"Speed up this scene"	**Video**	Adjust FFmpeg timing parameters for target clip segment
"Regenerate the script"	**Script**	Re-invoke LLM story phase with revised or same seed prompt

5.2 Intent Detection — How the Agent Knows What to Edit

An LLM-powered classification agent (built with LangGraph) receives the raw edit query and outputs a structured intent object:

```
{ "intent": "change_voice_tone", "target": "audio", "scope": "character:Narrator", "parameters": { "tone": "whispered" } }
```

The agent uses keyword extraction, semantic matching, and structured output parsing (Pydantic or JSON schema) to reliably classify each query into one of the following target categories:

- **audio** — Targets TTS output or background music. Triggers re-synthesis of affected audio segments only.
- **video_frame** — Targets still image generation for one or more scenes. Triggers re-run of the image generation step for those scenes.
- **video** — Targets the full compositing/export step. Triggers recomposition with updated parameters without regenerating assets.
- **script** — Targets the story/script output. Re-invokes Phase 1 and cascades changes through all downstream phases.

5.3 State Versioning & Undo

Every time the pipeline produces or modifies an output, the system saves a state snapshot. The snapshot includes all generated assets and the JSON state object at that moment. This enables full revert at any granularity:

- Each pipeline run or partial re-run increments a version counter (v1, v2, v3 …).
- The user can view a version history panel in the UI showing what changed at each version.
- Clicking Undo or selecting a previous version restores all assets and the JSON state to that snapshot.
- The state store uses a simple append-only log (SQLite or file-based), ensuring no version is ever permanently lost.

Recommended implementation pattern:

StateManager.snapshot(version, state_json, asset_paths) → persisted StateManager.revert(version) → restores assets + state + updates UI StateManager.history() → returns list of all versions with diff summary

The LangGraph agent uses a checkpointer (LangGraph's built-in MemorySaver or SqliteSaver) to persist the agent's own reasoning state across edit sessions, enabling coherent multi-turn editing conversations.

5.4 Deliverables

1. Edit intent classification agent (LangGraph node or chain) with test coverage across at least 10 edit query types.
2. State snapshot system with version history UI panel.
3. Undo/revert functionality that correctly restores both assets and the pipeline JSON state.
4. A short demo video or screen recording showing: initial generation → 3 edits → 2 reverts.

6. Division of Work

Each member owns one primary phase end-to-end, including its API integrations, data schema compliance, unit tests, and documentation. Member 4 additionally leads the editing agent.

Member	Primary Phase	Key Responsibilities
Member 1	Phase 1 — LLM Story & Script Pipeline	Prompt engineering, JSON schema design, LLM integration (local + API), character roster generation
	Phase 2 — Audio Generation & TTS	TTS integration (ElevenLabs/Coqui/Bark), voice consistency, music selection, timing manifest output
Member 2	Phase 3 — Video Generation & Composition	Stable Diffusion/DALL-E visuals, FFmpeg animation, A/V sync, subtitle overlay, MP4 export
Member 3	Phase 4 — Full-Stack App & Edit Agent	FastAPI/Django backend, React/Next.js frontend, LangGraph edit agent, state snapshot & undo system
Member 4	Phase 5 — Intelligent Edit & Version Management	OpenCV Filters, FFmpeg, Editing tools

All four members are jointly responsible for: (1) the shared JSON state schema, (2) integration testing across phase boundaries, and (3) the final project presentation and report.

7. Deliverables

5. Source code repository (GitHub) with clear folder structure per phase and a root README.
6. Working web application demonstrating the full pipeline (live demo or recorded walkthrough).
7. Phase-level unit tests demonstrating each module's inputs, outputs, and error handling.
8. Project report (8–12 pages) covering system design, architecture diagram, API choices, challenges, and results.
9. Final presentation (10–15 minutes) with live demo and Q&A.
10. Edit agent demo showing intent classification, execution, and undo in action.

8. Technology Stack Summary

Layer	Primary Options	Budget-Friendly Local Alt.
LLM / Agents	Claude API, GPT-4, LangGraph	Ollama + LLaMA 3 / Mistral
TTS	ElevenLabs API	Coqui TTS, Bark (local)
Image Gen	DALL-E 3, RunwayML	Stable Diffusion (local)
BGM	MusicGen (Meta)	Royalty-free library
Video Comp.	MoviePy + FFmpeg	FFmpeg only
Backend	FastAPI + Uvicorn	Django
Frontend	React + Vite	Next.js
State Store	SQLite (LangGraph SqliteSaver)	File-based JSON snapshots

9. Evaluation Criteria

Criterion	What We Look For	Weight
Phase 1 — Story & Script Agent	Correct JSON schema, coherent story, character consistency	**15%**
Phase 2 — Audio Generation	Voice consistency, audio quality, timing manifest accuracy	**15%**
Phase 3 — Video Composition	Visual quality, A/V sync, smooth transitions, final MP4	**20%**
Phase 4 — Web Interface	UX clarity, real-time progress, phase re-run capability	**10%**
Integration & Pipeline	All phases chain correctly; shared schema respected	**10%**
Report & Presentation	Design clarity, architectural depth, live demo quality	**10%**
Phase 5 — Edit Agent & Undo	Intent accuracy, correct targeted re-run, undo reliability	**20%**

10. A Note to Your Group

This is an ambitious project — intentionally so. Agentic AI systems are the direction the industry is moving, and building one from scratch, end-to-end, is exactly the kind of experience that sets graduates apart. The four phases are designed to be independently buildable, so even if one phase is incomplete, the others can mock its output and keep moving.

Start with Phase 1 together. Agree on the shared JSON schema in Week 1 before writing a single line of code in any other phase. Everything else depends on it.

Good luck — the goal is a system you are genuinely proud to demo.